The File Management System

BY

NAME: AJMAL
ROLL. NO: 45

NAME: Subhan Majeed
ROLL. NO: 18

NAME: M. Ehtisham Ashfaq
ROLL. NO: 25

DATE: 30/01/2026
VERSION: 1

Session 2023-2027

Department of Computer Science and Information Technology
Faculty of Computing and Engineering
University of Kotli Azad Jammu and Kashmir

Table of Contents

1. INTRODUCTION iv
2. OBJECTIVES v
2.1 Design of File Management System v
2.2 Implementation of File Operations v
2.3 Directory Management v
2.4 Data Integrity and Consistency v
2.5 User-Friendly Interface v
2.6 Educational and Practical Learning v
3. PROPOSED ALGORITHM vi
3.1 Initialization vi
3.2 Display Main Menu vi
3.3 User Input Selection vi
3.4 Execution of Selected Operation vi
3.5 Display Output Message vi
3.6 Repeat Process vi
3.7 5System Termination vi
4. FUNCTIONAL REQUIREMENTS vii
4.1 File Creation vii
4.2 File Reading vii
4.3 File Writing and Updating vii
4.4 File Deletion viii
5.5 File Renaming viii
4.6 Directory Creation viii
4.7 Directory Deletion ix
4.8 Display File and Directory Information ix
5. NON-FUNCTIONAL REQUIREMENTS x
5.1 Reliability x
5.2 Performance Efficiency x
5.3 Usability x
5.4 Portability x
5.5 Maintainability x

5.6 Security x
6. SYSTEM SCOPE xi
7. TOOLS AND TECHNOLOGIES xi
8. CONCLUSION xii
9. SUMMARY xii

1. INTRODUCTION

File management is a core function of every modern operating system, as it provides the mechanism for storing, organizing, accessing, and protecting data on storage devices. A well-designed file management system ensures that data is arranged in a logical manner, allowing users and applications to retrieve information efficiently and securely. With the rapid increase in digital data, effective file management has become essential for maintaining system performance, reliability, and data integrity.

A File Management System is responsible for handling various file-related operations such as creating files, reading and writing data, updating file contents, deleting files, and managing directories. These operations form the foundation of user interaction with computer systems. Without an efficient file management system, data would be difficult to locate, prone to loss, and vulnerable to unauthorized access. Therefore, understanding how file systems operate is an important aspect of computer science and software engineering education.

This project proposes the design and implementation of a File Management System using the C++ programming language. C++ provides powerful features for system-level programming and direct file handling, making it suitable for implementing file system concepts. The proposed system will simulate basic file and directory operations through a console-based, menu-driven interface. The system is intended to be simple, reliable, and easy to use, while still demonstrating the fundamental principles of file management.

The proposed File Management System is developed primarily for academic and learning purposes. It aims to provide students with hands-on experience in file handling, directory structures, and operating system concepts. By working on this project, students will gain a deeper understanding of how files are managed internally and how software interacts with storage devices. The project also enhances problem-solving skills, logical thinking, and practical programming experience in C++.

FILE MANAGEMENT SYSTEM

2. OBJECTIVES

The main objective of the proposed project is to design and implement a functional and efficient File Management System using the C++ programming language. The specific objectives of the study are described below.

2.1 Design of File Management System

The objective is to design a structured and logical File Management System that simulates basic file system operations. The design will focus on simplicity, clarity, and ease of understanding while maintaining core file management principles.

2.2 Implementation of File Operations

This project aims to implement essential file operations such as file creation, reading, writing, updating, deletion, and renaming. These operations will be developed using standard C++ file handling techniques to ensure correctness and reliability.

2.3 Directory Management

Another objective of the project is to manage directories efficiently. The system will allow users to create, delete, and navigate directories, helping to understand hierarchical file organization.

2.4 Data Integrity and Consistency

The project aims to ensure that data stored in files remains consistent and secure during file operations. Proper handling of file access and error conditions will be implemented to prevent data loss or corruption.

2.5 User-Friendly Interface

The system will be designed with a menu-driven interface to make it easy for users to interact with the system. This objective focuses on improving usability and reducing complexity for end users.

2.6 Educational and Practical Learning

The project aims to provide practical exposure to operating system concepts and file handling mechanisms. It is intended to strengthen students' understanding of system-level programming and enhance problem-solving skills using C++.

3. PROPOSED ALGORITHM

The proposed File Management System will follow a structured and step-by-step algorithm to ensure correct execution of file and directory operations. The algorithm is designed to be simple, efficient, and easy to understand.

3.1 Initialization

The system will start by initializing all required resources and variables. Necessary libraries for file handling and directory management will be loaded to prepare the system for execution.

3.2 Display Main Menu

After initialization, the system will display a menu containing all available operations such as creating files, reading files, writing data, deleting files, renaming files, managing directories, and exiting the program.

3.3 User Input Selection

The system will accept input from the user to select a desired operation from the menu. User input will be validated to ensure that only valid choices are processed.

3.4 Execution of Selected Operation

Based on the user's selection, the system will perform the corresponding file or directory operation using C++ file handling functions. Appropriate error handling mechanisms will be applied to manage invalid operations or system errors.

3.5 Display Output Message

After executing the selected operation, the system will display a success or error message to inform the user about the outcome of the operation.

3.6 Repeat Process

The system will return to the main menu after completing an operation. This process will continue until the user selects the exit option.

3.7 5System Termination

Once the exit option is selected, the system will safely release all allocated resources and terminate execution.

4. FUNCTIONAL REQUIREMENTS

Functional requirements define the specific actions and services that the File Management System must provide.

4.1 File Creation

The system shall allow users to create new files by specifying a file name and location. The system will ensure that duplicate file names are handled appropriately.

4.2 File Reading

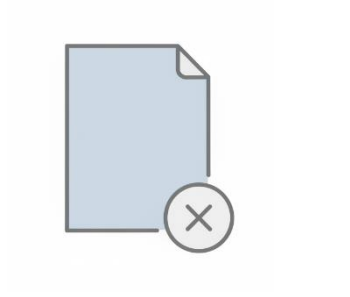
The system shall provide functionality to open and read the contents of existing files. Users will be able to view stored data in a readable format.

4.3 File Writing and Updating

The system shall allow users to write new data into files and update existing file contents. Proper file access modes will be used to prevent data loss.

4.4 File Deletion

The system shall enable users to delete files from storage. The system will verify the existence of a file before deletion to avoid invalid operations.

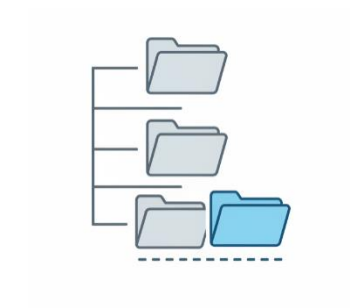


5.5 File Renaming

The system shall support renaming of existing files while maintaining their contents and storage location.

4.6 Directory Creation

The system shall allow users to create directories to organize files in a hierarchical structure.

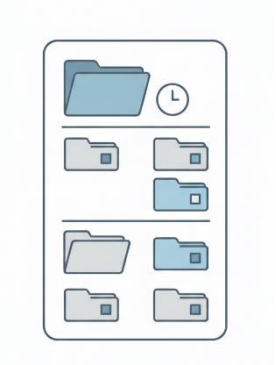


4.7 Directory Deletion

The system shall provide functionality to delete directories, ensuring that system constraints and safety checks are applied.

4.8 Display File and Directory Information

The system shall display relevant information such as file names, directory names, and basic attributes to help users manage stored data effectively.



5. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes and constraints of the File Management System. These requirements define how well the system performs its functions rather than what functions it performs.

5.1 Reliability

The system shall operate reliably and perform file operations without causing data loss or corruption. Proper error handling mechanisms will be implemented to ensure stable system behaviour.

5.2 Performance Efficiency

The system shall perform file and directory operations efficiently, ensuring minimal delay during execution. The design will aim to optimize file access and processing time.

5.3 Usability

The system shall be user-friendly and easy to operate. A menu-driven interface will be provided so that users with basic computer knowledge can use the system without difficulty.

5.4 Portability

The system shall be portable and capable of running on different platforms that support the C++ programming language, such as Windows and Linux.

5.5 Maintainability

The system shall be designed in a modular manner, making it easy to understand, modify, and maintain in the future.

5.6 Security

The system shall ensure controlled access to files and prevent unauthorized or accidental file operations as much as possible within the scope of the project.

6. SYSTEM SCOPE

The scope of the proposed File Management System is limited to the design and development of a console-based application for educational and academic purposes. The system focuses on simulating basic file and directory management operations such as file creation, reading, writing, updating, deletion, renaming, and directory handling using the C++ programming language. The project is intended to demonstrate fundamental file system concepts rather than replace or compete with real operating system file systems.

The system will operate within the constraints of standard file handling libraries provided by C++ and will run on platforms that support C++ compilation, such as Windows and Linux. Advanced features such as graphical user interfaces, network-based file sharing, multi-user access control, encryption, and recovery mechanisms are beyond the scope of this project. However, the proposed system provides a strong foundation that can be extended in the future to include more advanced functionalities.

The primary focus of the project is to enhance understanding of file management principles, directory structures, and system-level programming. The scope is deliberately kept simple to ensure clarity, reliability, and effective learning outcomes for students.

7. TOOLS AND TECHNOLOGIES

The development of the proposed File Management System will require a set of software tools and technologies that support system-level programming and file handling operations. The primary programming language used for this project will be C++, as it provides strong support for file input and output operations and is widely used for developing system-oriented applications.

A standard C++ compiler such as GCC or MinGW will be used to compile and execute the source code. The project will be developed and tested on commonly used operating systems such as Windows or Linux, both of which provide reliable support for C++ development and file system operations.

Standard C++ libraries including iostream will be used for input and output operations, fstream will be utilized for file handling tasks such as reading and writing data, and filesystem will be employed for directory management and file-related operations. A simple text editor or an integrated development environment will be used for writing and managing the source code.

These tools and technologies collectively provide a stable and efficient environment for implementing the proposed File Management System and achieving the objectives of the project.

8. CONCLUSION

The proposed File Management System project provides a practical and systematic approach to understanding fundamental file handling and operating system concepts. By designing and implementing the system using the C++ programming language, the project demonstrates how files and directories can be efficiently created, accessed, modified, and managed through programmatic techniques. The system simulates essential file management operations in a controlled environment, making it suitable for academic and learning purposes.

This project enhances practical knowledge of system-level programming, file structures, and directory management while strengthening logical thinking and problem-solving skills. The implementation of functional and non-functional requirements ensures that the system remains reliable, efficient, and user-friendly. Proper handling of file operations and error conditions also helps in understanding the importance of data integrity and system stability.

Furthermore, the File Management System serves as a strong foundation for exploring advanced operating system concepts. The project can be extended in the future to include features such as access control, file security, graphical user interfaces, and multi-user support. Overall, the successful completion of this project will contribute significantly to the academic development of students and prepare them for more complex system and software engineering challenges.

9. SUMMARY

This project presents the proposed design and implementation of a File Management System using the C++ programming language. The system focuses on simulating fundamental file and directory management operations, including file creation, reading, writing, updating, deletion, renaming, and directory handling. The project has been structured to provide a clear understanding of file system concepts and operating system fundamentals.

The proposal outlines the objectives, algorithm, functional and non-functional requirements, system scope, and tools and technologies required for development. The File Management System is intended primarily for academic and learning purposes, offering practical exposure to system-level programming and file handling techniques. The successful implementation of this project will enhance technical skills and provide a solid foundation for further exploration of advanced file system and operating system concepts.

10.REFERENCES

1. **File Management System Emulator (GitHub)** – A file management system inspired by Linux shell, written in C++. It supports basic directory and file operations and serves as a learning resource for system-level programming.
   https://github.com/abhishekmj303/FileSystem-Emulator

2. **File-Manager (GitHub)** – A simple cross-platform file manager application implemented in C++ using the Qt framework. This project can be studied to understand GUI-based file handling in C++.
   https://github.com/kitswas/File-Manager

3. **Simple File Management System (SourceCodester)** – A basic file management system project written in C++ with source code available for download, demonstrating simple operations like file creation, display, and deletion.
   https://www.sourcecodester.com/cc/16737/simple-file-management-system-c-free-source-code.html

4. **Simple File Management System (Alternative Source)** – Another C++ file management project with free source code for students to learn file handling and basic operations.
   https://www.sourcecodester.com/cc/14270/simple-file-management-system.html

5. **Simple File Management System using C++ (SourceCodester alternative)** – A simple downloadable project that illustrates basic file manipulation tasks in a console program.
   https://www.sourcecodester.com/cc/15011/simple-file-management-system-using-c-free-source-code.html